

Hooks previos a la herramienta y ampliar el espacio de trabajo

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES

Sesión 6 de 12 · Notas en español (es-ES)

Los hooks previos a la tool corren *antes* de que Edit, Write o Bash aterricen. La sesión 6 los usó para excluir rutas protegidas y denegar comandos peligrosos — sobre todo con el modo de permiso en auto. La misma hora mostró cómo **ampliar el espacio de trabajo** más allá del markdown de un repo: instalar plugins del marketplace (Playwright y similares), o empaquetar vuestros agentes, hooks y skills como plugin de Claude Code (`plugin.json` + manifiesto de marketplace) para que otros proyectos hereden el toolkit.

LOS NOMBRES DE CARPETA DEL PLUGIN SE MOVERÁN

El demo usó esquemas `.claude-plugin / marketplace.json / plugin.json`, instalación usuario frente a proyecto frente a repo local, y un paquete «To Do Toolkit». Los errores de sintaxis (ruta source incorrecta, referencias circulares, necesidad de recargar) comieron mucho tiempo en directo. Copiad la estructura de los docs vigentes de [Claude Code](#), no del ensayo y error de la grabación.

Pre-tool use: denegar antes del daño

Aclaración de Rajesh: bloquear `rm` es un hook **pre**, no post. Cuando el fichero ya no está, Prettier no ayuda. El `settings.json` del formador tenía *varios matchers* bajo pre-tool-use: uno hacia `protect-files.sh`, otro hacia `block-dangerous-commands.sh`. Los matchers eligen qué tool (Write frente a Bash) lanza qué script.

- Protect-files: no tocar `.env`, `package-lock.json`, `.git`, ficheros secretos de ORM (Prisma fue el ejemplo), y lo que añadáis.
- Comandos peligrosos: patrones que no dejáis correr al shell. Lalit señaló que `git restore / revert` pueden borrar contenido sin el token `rm`: ampliád la lista negra, no asumáis que basta un nombre de binario.
- Modo auto: «Claude no preguntará si actualizar un fichero o ejecutar un comando». Los hooks son el freno que queda.
- Los subagentes heredan la misma necesidad: un revisor desatendido con Write sigue sin poder tocar secretos.

Cuando el hook «no dispara»

El demo en directo de `rm test.txt` falló hasta: recargar VS Code, recargar webviews (Claude es un componente web en el IDE), corregir el nombre de fichero y probar desde el cliente de **terminal**. Aun así apareció un error de hook no bloqueante. Hubo que activar una función preview («chat is the Claude hook») para que la ventana de chat respetara los ajustes del repo. Conclusión de examen: la configuración no está

completa hasta que la misma superficie que usáis de verdad haya ejecutado el matcher. Python o cualquier cosa en shell está permitido; el momento lo marca el evento, no el lenguaje.

Plugins de marketplace como extensión instantánea del espacio de trabajo

Los plugins instalan un conjunto de hooks, skills y agentes en el espacio de trabajo de Claude. Ámbito: usuario, este proyecto o repo local. Ejemplo: un plugin oficial de Playwright — tras instalar, settings lo listaba; las skills/comandos pueden exigir recargar el terminal para verse. Los plugins de diseño frontend inyectan paquetes de prompt/skill de forma parecida. Si un plugin no trae skills propias, seguís teniendo los agentes/hooks que embarcó. Así un equipo normaliza «qué hooks debe haber» sin copiar ficheros de shell a mano en cada repo greenfield.

Montar vuestro propio plugin (To Do Toolkit)

1. Reunid agentes, hooks y skills personalizados bajo una carpeta toolkit.
2. Añadid el manifiesto del plugin (name, display name, version, description, keywords, author) según el esquema de plugin de Claude Code.
3. Añadid un marketplace.json (o el equivalente vigente) que apunte a la ruta source del plugin sin referencias circulares.
4. Colocad los ficheros bajo la carpeta punto que los docs nombren esa semana.
5. En Claude Code: manage plugins → añadir marketplace o importar la URL de la carpeta → instalar a nivel de proyecto o de usuario.
6. Confirmed en settings.json que el plugin está referenciado; los slash (p. ej. una skill backend del toolkit) aparecen sin ficheros nuevos en el árbol de la app.

Mritunjay: un plugin es una biblioteca que pasáis a otros; no pueden cambiarla salvo que tengan acceso (colaborador/contributor de GitHub). Los artefactos instalados viven en el sandbox (usuario frente a proyecto). Eso es «ampliar el espacio de trabajo» en el sentido Foundation: el repo vacío más un plugin ya es un entorno Claude Code con opinión.

Por qué sale en los escenarios de examen

El escenario 2 de la lista posterior del formador es literalmente Claude Code para desarrollo de software: CLAUDE.md, slash personalizados, plan mode frente a ejecución directa. Los hooks previos y los plugins son las respuestas de capa de configuración cuando el escenario añade «modo auto», «no subáis secretos» o «desplegad esto en 50 repos». Otros escenarios (agente de soporte, investigación multiagente, CI/CD, extracción estructurada) siguen asumiendo que conocéis allow/deny de tools: los hooks previos son la versión nativa de Claude Code de esa idea.

UNA CADENA PARA RECORDAR

Ask/plan mientras aprendéis → auto solo cuando las listas de denegación pre-tool funcionan en vuestro cliente real → formato con comandos locales post-tool → distribuir el conjunto como plugin.

CLAUDE.md se queda para el significado; los hooks, para la mecánica.

Logística de aula (no es producto, pero se preguntó en la sesión)

Antes de que Vijay empezara, facilitación repitió: no compartáis pantalla con datos confidenciales; los portátiles UST pueden seguir bloqueando Pearson VUE — examen en un ordenador personal o en un centro. Es una restricción de entrega, no un ajuste de Claude Code.

Documentación oficial

- **Visión general de Claude Code**

<https://code.claude.com/docs/en/overview>

Matchers de hooks, plugins, marketplaces, instalación usuario frente a proyecto.

- **MCP**

<https://docs.claude.com/en/docs/agents-and-tools/mcp>

Los plugins no son servidores MCP; embarcan skills/agentes/hooks. MCP sigue siendo para tools de terceros.

- **Guardrails**

<https://docs.claude.com/en/docs/test-and-evaluate/strengthen-guardrails>

Los scripts de denegación pre-tool son guardrails en el cliente de código.

- **Anthropic Academy (Skilljar)**

<https://anthropic.skilljar.com>

Confirmad pesos de dominio y los seis escenarios de producción antes del examen.

- **Inicio rápido de Managed Agents**

<https://platform.claude.com/docs/en/managed-agents/quickstart>

Extensión del espacio de trabajo en plataforma cuando dejáis el modelo de plugin del IDE.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.